



Exercise 1 : Class and Sequence Diagrams

The classes `Banque`, `Client` and `Compte` are provided to simulate, in a highly simplified way, how a bank operates. These classes are tested using the `main` defined in the file `main.cpp`.

```
/**
 * @file banque.hpp
 * @author Michel Ianotto
 *
 * @brief Declaration of the Banque class
 */
#pragma once

#include <vector>
#include <memory>
#include <iostream>
#include <string>

#include <compte.hpp>

class Banque {

private:
    std::vector<std::shared_ptr<Compte>> lesComptes;
    static int numCompte;

public:
    Banque();
    void retraitImpossible(int numeroCompte, double montant);
    void debiter(int numeroCompte, double montant);
    int creerCompte(double soldeInit);
    std::shared_ptr<Compte> rechercherCompte(int numero);
};
```

```
/**
 * @file banque.cpp
 * @author Michel Ianotto
 *
 * @brief Implementation of the Banque class methods
 */
#include <iostream>
#include <algorithm>

#include <banque.hpp>
```

```

int Banque::numCompte = 0;

Banque::Banque() {
}

void Banque::retraitImpossible(int numeroCompte, double montant) {
    std::cout << "Insufficient balance for the withdrawal of "
                << montant
                << "€ from account no. "
                << numeroCompte
                << std::endl;
}

std::shared_ptr<Compte> Banque::rechercherCompte(int numero)
{
    for (const auto& compte : lesComptes) {
        if (compte->donneNumero() == numero) {
            return compte;
        }
    }
    return nullptr;
}

void Banque::debiter(int numeroCompte, double montant) {
    std::shared_ptr<Compte> compteClient;
    double solde;

    compteClient = rechercherCompte(numeroCompte);
    solde = compteClient->donneSolde();

    if (solde > montant) {
        compteClient->debiterCompte(montant);
        std::cout << "Withdrawal of "
                    << montant
                    << "€ effectué from account no. "
                    << numeroCompte
                    << std::endl;
    }
    else
        retraitImpossible(numeroCompte, montant);
}

int Banque::creerCompte(double soldeInit) {
    lesComptes.push_back(
        std::make_shared<Compte>(
            numCompte,
            soldeInit));

    return numCompte++;
}

/**
 * @file client.hpp
 * @author Michel Ianotto
 *
 * @brief Declaration of the Client class
 */

```

```

#pragma once

#include <banque.hpp>

class Client {

private:
    std::string nom;
    int monNumero;
    std::shared_ptr<Banque> maBanque;

public:
    Client(const std::string & unNom);
    void retrait(double montant);
    void ouvrirCompte(std::shared_ptr<Banque>, double);
};

/**
 * @file client.cpp
 * @author Michel Ianotto
 *
 * @brief Implementation of the Client class methods
 */
#include <iostream>
#include <memory>
#include <string>

#include <client.hpp>
#include <banque.hpp>

Client::Client(const std::string& unNom) : nom(unNom), monNumero(0) {

}

void Client::retrait(double montant) {
    maBanque->debiter(monNumero, montant);
}

void Client::ouvrirCompte(std::shared_ptr<Banque> uneBanque, double soldeInit) {
    maBanque = uneBanque;
    monNumero = maBanque->creerCompte(soldeInit);
    std::cout << "Client_"
                << nom
                << "_opened_account_no_"
                << monNumero
                << "_with_an_initial_balance_of_"
                << soldeInit << "€"
                << std::endl;
}

/**
 * @file compte.hpp
 * @author Michel Ianotto
 *
 * @brief Definition of the Compte class
 */
#pragma once

```

```

class Compte {
protected:
    int numero;
    double solde;

public:

    Compte(int unNumero, double soldeInit) : numero(unNumero), solde(soldeInit)
    {};
    virtual ~Compte() {};
    virtual void debiterCompte(double montant) {
        solde = solde - montant;
    };
    double donneSolde() const { return solde; };
    int donneNumero() const { return numero; };
};

```

```

/**
 * @file main.cpp
 * @author Michel Ianotto
 *
 * @brief Programme principal de l'application qui gère les comptes bancaires
 */

#include <memory>

#include <client.hpp>
#include <banque.hpp>

int main() {
    auto banqueNationale = std::make_shared<Banque>();
    auto c1 = std::make_shared<Client>("C1");

    c1->ouvrirCompte(banqueNationale, 1000);
    c1->retrait(200);

    return 0;
}

```

Question 1 Compile the files `banque.cpp`, `client.cpp` and `main.cpp` using the following command :

```
g++ -std=c++20 -I. -o main main.cpp banque.cpp client.cpp
```

and run the `main`.

Question 2 : Using a sequence diagram, describe the messages exchanged between the objects when executing the statement `c1->retrait(200)` in the `main` function. Indicate the objects' periods of activity.

Question 3 : Provide the class diagram including the classes `Client`, `Banque` and `Compte`. Indicate the types of the attributes and the signatures of the methods.

The software design is now modified to handle both savings accounts and current accounts. The savings account has an interest rate and does not allow overdrafts, while the current account allows overdrafts (up to a certain limit). The method `debiterCompte` is modified to return "true"

if the debit is possible and “false” otherwise (if it would result in a negative balance on a savings account, or exceed the authorized overdraft limit on a current account). **We assume that a customer has either a current account or a savings account.**

Question 4 : Provide the class diagram involving the classes `Client`, `Banque`, `Compte`, `CompteEpargne` and `CompteCourant`.

Question 5 : Implement the `CompteCourant` and `CompteEpargne` classes and modify the `Banque` and `Compte` classes so that executing the `main` function below produces the following output :

```
Client C1 opened savings account no. 0 with an initial balance of 1000€
Withdrawal of 200.0€ effectué from account no.  : 0
Client C1 opened checking account no. 1 with an initial balance of 500€
Insufficient balance for the withdrawal of 700€ from account no.  1
```

Exercise 2 : Recursive Aggregation for Managing Geometric Figures

We want to develop a simplified vector drawing application. A drawing consists of geometric shapes. A geometric shape can be :

- a circle ,
- a rectangle ,
- a group of shapes.

The goal is to manage all geometric shapes as a tree in order to apply geometric transformations (translation, rotation, ...) to shapes located at a given depth in the tree.

Question 1 : Propose a class diagram for manipulating these geometric shapes. For this purpose, we propose defining :

- a base class `Forme`,
- two leaf classes `Rectangle` and `Cercle` that inherit from the class `Forme`,
- a class `Groupe` that inherits from the class `Forme` and contains a vector of `Forme` objects

In each class, define the `afficher` method, which produces an indented textual representation of the tree, useful in particular for debugging, and an `aire` method, which computes the area of a geometric shape.

Question 2 : Implement the classes from the class diagram proposed in the previous question and test the classes with the following `main` function :

```
#include <groupe.hpp>
#include <rectangle.hpp>
#include <cercle.hpp>

#include <iostream>
#include <memory>

int main()
{
    // Construction of the geometric-shape tree.
    // The variable dessin represents the root of the tree
    auto dessin = std::make_shared<Groupe>();

    // Construction of a group of geometric shapes
```

```

auto groupe1 = std::make_shared<Groupe>();
// Construction of a circle with radius 10 and adding it to groupe1
groupe1->ajouter(std::make_shared<Cercle>(10));
// Construction of a circle with radius 15 and adding it to groupe1
groupe1->ajouter(std::make_shared<Cercle>(15));
// Adding groupe1 to the drawing
dessin->ajouter(groupe1);
// Construction of a rectangle with length 40 and width 20 and adding
// the rectangle to the drawing
dessin->ajouter(std::make_shared<Rectangle>(40,20));

// Displaying the geometric shapes
dessin->afficher();

// Computing and displaying the total area of the geometric shapes
std::cout << "Total_area="
    << dessin->aire()
    << std::endl;

return 0;
}

```

Executing the `main` must produce the following output :

```

Groupe
  Groupe
    Cercle (r = 10)
    Cercle (r = 15)
  Rectangle (longueur = 40, largeur = 20)
Total area = 1821.02

```

Question 3 : Propose a sequence diagram showing the messages exchanged between the objects when executing the statement `dessin->aire()`. Create the class diagram using **Visual Paradigm**. Clearly indicate the objects' periods of activity.

We now want to save the geometric shapes in a file in **svg** (Scalable Vector Graphics). Each group of shapes has its own geometric transformation : translation (**tx**, **ty**) and rotation (**angle**). A Rectangle is defined by its length and width and by the coordinates (x,y) of its upper-left corner. A circle is defined by its radius and the coordinates (cx, cy) of its center. For example, the following geometric-shape tree :

```

Groupe
  (tx = 10, ty = 10, angle = 0)
  Groupe
    (tx = 100, ty = 50, angle = 0)
    Cercle (r = 10) (cx = -20, cy = 0)
    Cercle (r = 15) (cx = 20, cy = 0)
  Rectangle (longueur = 80, largeur = 20) (x = 0, y = 0)

```

must produce the following **svg** suivant :

```

<svg xmlns="http://www.w3.org/2000/svg">
<g transform="translate(10,10) rotate(0)">
<g transform="translate(100,50) rotate(0)">
<circle cx="-20" cy="0" r="10" fill="red" />
<circle cx="20" cy="0" r="15" fill="red" />
</g>
<rect x="0" y="0" width="80" height="20" fill="blue" />
</g>

```

</svg>

and the display below :

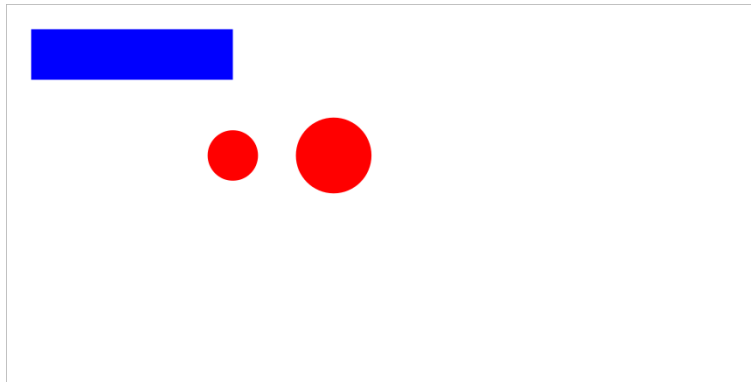


FIGURE 1 – Display of the SVG file

Question 4 : Modify the code of the classes `Forme`, `Rectangle`, `Cercle` et `Groupe` to save the geometric shapes to an `svg`.

Question 5 : Test your classes with the following `main` ci-dessous :

```
#include <groupe.hpp>
#include <rectangle.hpp>
#include <cercle.hpp>

#include <iostream>
#include <memory>

int main()
{
    // Construction of the geometric-shape tree
    // dessin represents the root of the tree
    auto dessin = std::make_shared<Groupe>(10, 10, 0);
    // Construction of a group of geometric shapes
    auto groupe1 = std::make_shared<Groupe>(100, 50, 0);
    // Construction of a circle with radius 10 and center -20, 0 and adding it to
    // groupe1
    groupe1->ajouter(std::make_shared<Cercle>(10, -20, 0));
    // Construction of a circle with radius 15 and center 20, 0 and adding it to
    // groupe1
    groupe1->ajouter(std::make_shared<Cercle>(15, 20, 0));
    // Adding groupe1 to the drawing
    dessin->ajouter(groupe1);
    // Construction of a rectangle with width 80 and height 20 with an
    // upper-left corner at coordinates (0,0) and adding the rectangle to the
    // drawing
    dessin->ajouter(std::make_shared<Rectangle>(80, 20, 0, 0));
    // Displaying the geometric shapes
    dessin->afficher();

    std::cout
        << std::endl;

    // Computing and displaying the total area of the geometric shapes
```

```

std::cout
    << "Total area = "
    << dessin->aire()
    << std::endl;

// Exporting the drawing to an SVG file
std::ofstream fichier;
fichier.open("dessin.svg");

fichier
    << "<svg xmlns=\"http://www.w3.org/2000/svg\">"
    << std::endl;
dessin->exportSVG(fichier);
fichier << "</svg>";

fichier.close();
}

```

Advanced Question : The C++ code for the `ExportSVGVisitor` class and the `main` function is provided below. The `ExportSVGVisitor` class contains the functions used to save geometric shapes to an SVG file. Integrate these two files into your project and modify the `Forme`, `Cercle`, `Rectangle`, and `Groupe` classes so that executing the `main` function produces the result shown below :

```

<svg xmlns="http://www.w3.org/2000/svg">
<g transform="translate(10,10) rotate(0)">
<g transform="translate(100,50) rotate(0)">
<circle cx="-20" cy="0" r="10" fill="red" />
<circle cx="20" cy="0" r="15" fill="red" />
</g>
<rect x="0" y="0" width="80" height="20" fill="blue" />
</g>
</svg>

```

```

#pragma once

#include <fstream>

class Cercle;
class Rectangle;
class Groupe;

class ExportSVGVisitor {
private:
    std::ofstream fichier;

public:
    ExportSVGVisitor(const std::string& nom);
    ~ExportSVGVisitor();
    void exporter(Cercle&);
    void exporter(Rectangle&);
    void exporter(Groupe&);
};

#include <exportSVGVisitor.hpp>

#include <cercle.hpp>
#include <rectangle.hpp>

```

```

#include <groupe.hpp>

ExportSVGVisitor::ExportSVGVisitor(const std::string& nom) {
    fichier.open(nom);

    fichier
        << "<svg xmlns=\"http://www.w3.org/2000/svg\">"
        << std::endl;
}

ExportSVGVisitor::~ExportSVGVisitor() {
    fichier << "</svg>";
    fichier.close();
}

void ExportSVGVisitor::exporter(Cercle& c) {
    fichier
        << "<circle cx=\""
        << c.getCx()
        << "\" cy=\""
        << c.getCy()
        << "\" r=\""
        << c.getRayon()
        << "\" fill=\"red\"/>"
        << std::endl;
}

void ExportSVGVisitor::exporter(Rectangle& r) {
    fichier
        << "<rect x=\""
        << r.getX()
        << "\" y=\""
        << r.getY()
        << "\" width=\""
        << r.getLongueur()
        << "\" height=\""
        << r.getLargeur()
        << "\" fill=\"blue\"/>"
        << std::endl;
}

void ExportSVGVisitor::exporter(Groupe& g) {
    fichier << "<g transform=\"translate("
        << g.getTx() << "," << g.getTy() << ") rotate("
        << g.getAngle() << ")\">\n";

    for(auto const& f : g.getFormes()) {
        f->accepter(*this);
    }
    fichier << "</g>\n";
}

```

```

/*
 * g++ -g -std=c++20 -I. -o main main.cpp cercle.cpp groupe.cpp rectangle.cpp
 * exportSVGVisitor.cpp
 */

```

```

#include <groupe.hpp>
#include <rectangle.hpp>

```

```

#include <cercle.hpp>
#include <exportSVGVisitor.hpp>

#include <iostream>
#include <memory>

int main()
{
    // Construction de l'arborescence de formes géométriques
    // dessin représente la racine de l'arborescence
    auto dessin = std::make_shared<Groupe>(10, 10, 0);

    // Construction d'un groupe de forme géométriques
    auto groupe2 = std::make_shared<Groupe>(100, 50, 0);
    groupe2->ajouter(std::make_shared<Cercle>(10, -20, 0));
    groupe2->ajouter(std::make_shared<Cercle>(15, 20, 0));
    dessin->ajouter(groupe2);
    dessin->ajouter(std::make_shared<Rectangle>(40, 20, 0, 0));
    dessin->afficher();

    std::cout
        << std::endl;

    std::cout
        << "Aire totale = "
        << dessin->aire()
        << std::endl;

    ExportSVGVisitor exportSVG("dessin.svg");

    dessin->accepter(exportSVG);
}

```

